

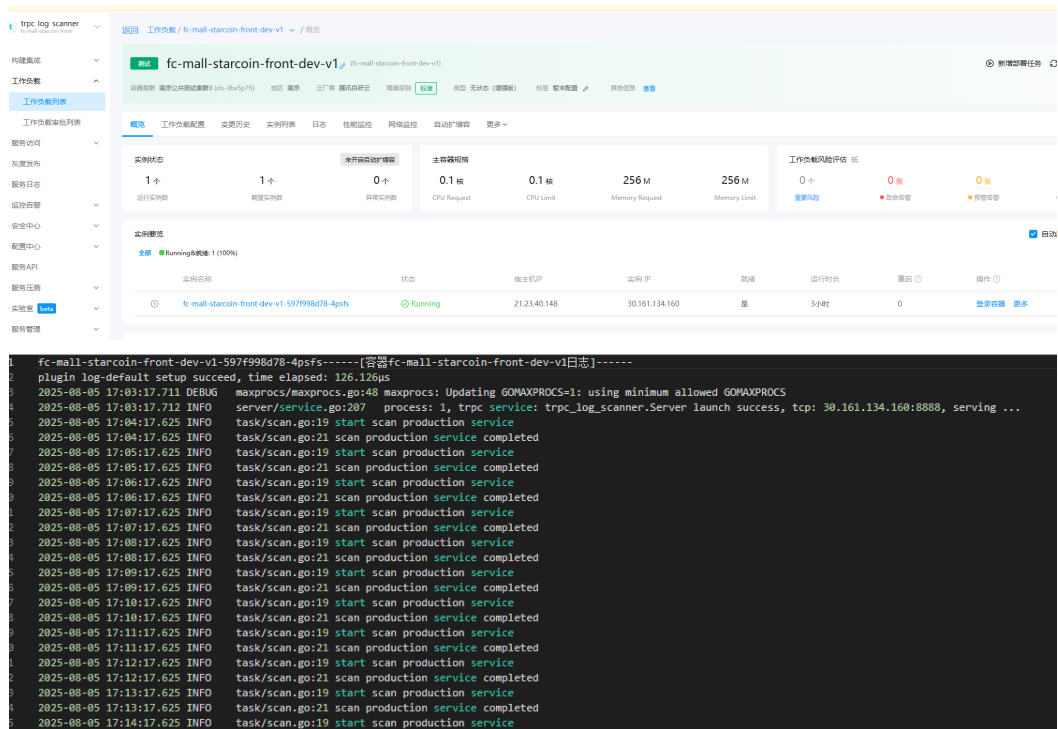
20250805日报

今日工作内容

1. 研究“线上go+python服务正式环境不允许debug日志级别”需求；
 - a. 当前进度：
 - i. 新启一个gdp服务，此服务是一个空服务，它只是做一个间隔为一小时的扫描告警功能，具体描述为：启了一个空服务（因为这个需求我觉得应该没有上游调用），然后在协程里每一个小时触发一次扫描（即拉接口），遍历所有正式服部署下的服务，每个服务都去判断配置文件中的日志级别，如果是debug级别就电话告警；
 - ii. 扫描逻辑代码：

```
1  // main.go
2  type serverImp struct{}
3
4  func main() {
5      s := trpc.NewServer()
6
7      trpc_log_scanner.RegisterServerService(s, &serverImp{})
8      if err := s.Serve(); err != nil {
9          log.Fatal(err)
10     }
11 }
12
13 // scan.go
14 func init() {
15     ctx, cancel := context.WithCancel(context.Background())
16     ticker := time.NewTicker(1 * time.Hour) // 每小时触发
17     go func() {
18         defer ticker.Stop()
19         defer cancel()
20         for {
21             select {
22             case <-ticker.C:
23                 log.InfoContext(ctx, "start scan production service")
24                 scan()
25                 log.InfoContext(ctx, "scan production service
26 completed")
27             case <-ctx.Done():
28                 return
29             }
30         }
31     }()
32 }
```

gdp端：



（这里我是以每一分钟扫描做的测试）。

iii. 后续要等gdp那边接口开发完成后再进行后续开发，需要在judgeandreport()中调用接口，遍历每个服务的配置文件，并判断是否需要告警；

2. 调研分布式服务中保障服务稳定性的技术；

- a. 降级监控；
- b. 自动化熔断；
- c. 自动化故障转移：当节点故障时，实时更新服务列表，流量自动迁移至健康节点；
- d. 动态负载均衡：采用加权最少连接算法，优先将请求分配给负载较低的节点，可以结合Ngix实现；
- e. 节点级限流：通过网关设置节点QPS；

明日待办：

- 1. 明天和强哥沟通“总控服务稳定性提升-样本架构优化避免大节点流量过大导致样本缺少”需求后，再开始开发；